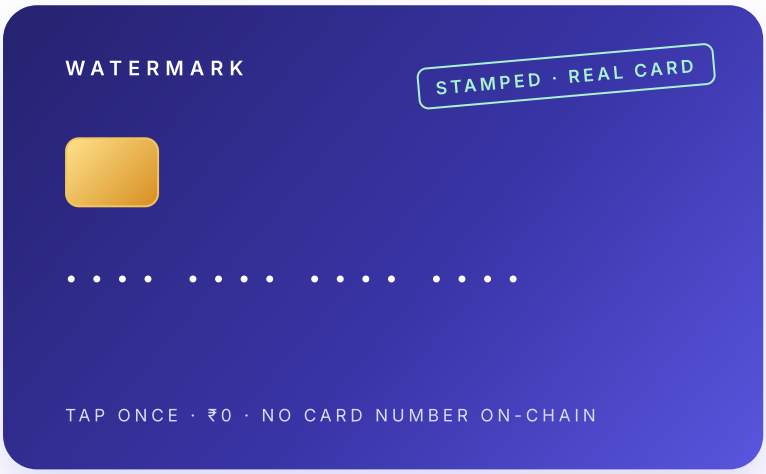
Watermark

Build spec, A to Z

Everything a developer, or Claude Code, needs to build Watermark in 36 hours: architecture, exact interfaces, the card maths, the privacy limits, a spec for every part, the team split, git rules and the submission checklist.



Proven on real cards

2 RuPay cards: full chain verified with the network key, and the card signed our random number live. SBI debit still needs its own test.

Two main tracks

Whitechain Scale ($3k) and the EF alt mempool track ($5k). Arkiv ($4k) is optional.

Golden rule

Claude Code never pushes. Prashant reviews and pushes every time.

> **One line: tap your bank card on your phone once, and your wallet gets a private "real card" stamp that bots can't fake, with no money charged and no card number on-chain.**

Project name: Watermark. This replaces every earlier name.

Deadline: Saturday 7 Nov, 9:00 AM IST

CONTENTS

0 How to use + golden rules
0a What changed in v3
A The product in one page
B What we proved on real cards
C System architecture
C2 Data and privacy map
D Repo layout and tooling
E Git workflow + CLAUDE.md
F Interfaces between the parts
G Card layer (EMV over NFC)
H The verification maths
I ZK circuit spec
I2 Filters (bank, network, type)
J Smart contracts
K Relayer and drain protection
L EF alt mempool (EIP-8141)
M Android Watermark Tap app
N Website
O Prover service
P MCP server and SDK
Q Arkiv (optional)
R Team split and timeline
S Testing plan
T Demo script
U Submission checklist
V Risks and fallbacks
W Links

0 · HOW TO USE THIS DOC

Read this first

Humans: read 0a, A, B, C, C2 and R. Then jump to the section for the part you own.

Claude Code: paste section E's CLAUDE.md into the repo root first. Then give it one section at a time (for example "build section M") together with section F, because F holds the interfaces every part must match.

"Verified" means we tested it on real cards, or checked official docs or the live chain, on 5 Nov. **"Check at kickoff"** means confirm it on the day before building on it.

Golden rules (non-negotiable)

1. **Claude Code never runs** `git push`, on any branch or remote. Prashant reviews and pushes manually, every time.
2. No force-push, no history rewrites, no remote changes, no `git reset --hard` on shared branches.
3. **Real card data never enters the repo:** no card numbers, raw logs, certificates from real cards, or notes linking a BIN to a team member's card. Tests use the synthetic test card (section S).
4. No secrets in git: `.env`, private keys, keystores and access keys stay local. Commit `.env.example` only.
5. **All project code is written after kickoff,** committed in small steps, and AI use is logged in `docs/AI_USAGE.md`. Card Test v2 is pre-kickoff research: don't copy it into the repo, and disclose it in the README.
6. **Never fake a step or a claim.** Anything estimated or untested is labelled that way on screen and in the README.

Before kickoff: ask the organisers

ETHGlobal's Classic track bars prior project-specific code, designs and assets, and a project started before the event loses partner and Finalist eligibility. Ask: "We tested card feasibility with a small app and wrote a plan before kickoff. We will rewrite all code after kickoff and disclose both. Is that OK?" Follow their answer over this doc.

0A · WHAT CHANGED IN V3

Build from this version, not v2

Area	Problem in v2	What v3 does
Name	Old project name throughout	Everything is Watermark: contracts, app, deep link, MCP tools, Arkiv type.
Existing app	Spec reused pre-kickoff code	New code after kickoff. Card Test v2 is disclosed research only.
Privacy claim	"No card number leaves your phone" was false	Say "no card number on-chain". The prover briefly sees card data, and we say so.
Card data in transit	Plain HTTP on the LAN	Phone encrypts with the prover's built-in public key.
Fake QR	QR could send card data anywhere	A fake prover gets only ciphertext. The session is signature-checked and the wallet is shown first.
Sessions	`openSession` cost gas, so bots could drain the relayer	Relayer signs sessions off-chain. No gas is spent until a valid proof exists.
Expiry	Card expiry public; date unchecked by the contract	The proof shows "valid until month X" only. The contract checks X. Stamps last about 3 months.
Renewal	Expired stamp locked the card out forever	Same card + same wallet can renew. A different wallet is refused.
Public inputs	Keccak values may not fit a field	`appId` is 248 bits and `un` is a number (F4).
Proof mode	ZK mode not specified	Pin one `bb` version and confirm real ZK mode. A non-ZK proof could expose the card number.
Filters	6-digit BIN only; network filter not bound	Leaves carry network + 6 or 8 digit prefix. Rules are immutable per app. Phone checks first.
Devnet	"Same proof" fails: it's tied to one session	A second tap with a devnet session.
Quota paymaster	Users could skip `consume()`	VERIFY rejects a transaction without the consume frame. Tested by trying to skip it.
Bot demo	1,000 real sends ≈ 0.33 WBT (rough, at 65k gas); faucet gives 0.5/day	Protection off: 25 real sends + 975 labelled estimates. Protection on: 1,000 real refusals.
USDT	No source for a new wallet's USDT	Ask Whitechain in hour 0. A business wallet funds the user.
Demo cards	Generic	SBI debit = live. Pine Labs metro cards = test only, and refused on the live policy.
MCP	Read-only checks only	Adds filters and policy drafting. It holds no keys and never takes card data.
Repo location	Current git root is the home folder, which holds `.ssh`	Fresh, separate repo. Check the root before any commit.

Checked live on 5 Nov

Whitechain RPC returns chain 1874. Alt mempool RPC returns chain 1337. The USDT address matches the official prize page, and its implementation contains `permit` and `transferWithAuthorization` (selector check; no transfer sent yet). None of the new design has been built or run yet.

A · THE PRODUCT IN ONE PAGE

What we're building

Problem

A new wallet is free and takes one second, so bots create thousands. Gasless apps get their gas budget drained, airdrops get farmed, and faucets get emptied. World ID needs an Orb; ID-based proofs need Aadhaar or a passport.

Solution

Tap a bank card on an Android phone. The chip signs a fresh challenge. A ZK proof shows "genuine network-signed card, passes this app's rules, first time here" without putting the card on-chain. The wallet gets a stamp any contract can check.

The four promises (what the build must deliver)

Promise	How the build delivers it
Bots out	Only a real chip with a key signed by a supported network passes the proof (section H). Verified on RuPay; Visa and Mastercard are untested.
One card, one stamp per app	Nullifier = Poseidon2(card public key, appId), stored on-chain and tied to one wallet (sections I and J).
No card number on-chain	Only the proof and public inputs go on-chain. Card data is encrypted from phone to prover, used in memory, then wiped (sections C2, M and O).
₹0, no payment	On our RuPay path we stop at GET PROCESSING OPTIONS and READ RECORD. Other networks may need GENERATE AC or INTERNAL AUTHENTICATE. Nothing is ever sent to a bank (section G).

What a stamp means: "this wallet was linked to a real bank card that passed this app's filter." It does not mean one unique person, KYC, age or balance. Never claim those.

Partner tracks (verified on the ETHGlobal Mumbai prize page)

Track	Prize	What we must show
Whitechain: Scale, Payments Nobody Notices	$3,000 (2k / 1k)	Zero visible crypto steps for the payer; official USDT settled in the main flow (self-minted tokens don't count); contracts verified on chain 1874; how the business pays gas, how the relayer gets WBT, what stops draining; a README line on why on-chain settlement is needed; `WHITECHAIN_FEEDBACK.md`.
EF: Best Use of the Frame Transaction Alternative Mempool	$5,000 (2.5k / 1.5k / 1k)	A link to an admitted transaction on the devnet explorer, or code that submits one; explain the design and trade-offs. One EF prize per project: you can submit to several EF tracks but get placed in at most one.
Arkiv: Build on Arkiv (from scratch), optional	$4,000 (3k / 1k)	Writes on Arkiv testnet plus a user-visible read; `/arkiv/` docs (schema, friction, reach, why-arkiv, how-you-use-arkiv); writer address and a runnable query in the README.

ETHGlobal rules that bind us

- Up to 3 partner prizes; several tracks from one partner count as one pick.
- Large single commits or missing history may mean disqualification. Commit steadily.
- Document where AI was used, and include all specs, prompts and planning files. AI must assist, not replace, the team's own work.
- Finalist judging: 4 minute demo + 3 minutes Q&A, scored on technicality, originality, practicality, usability and wow factor.

Verified facts from 2 real RuPay cards

Tested on 5 Nov 2026 with the Card Test v2 app, then checked with a separate maths script. Card numbers and BINs are intentionally not shown.

Item	Value seen on both cards
Payment app (AID)	`A0000005241010` (RuPay). One card is co-badged with Discover, `A0000001523010`.
PDOL (data the card asks for at GPO)	`9F40(5) DF3A(5) 9F33(3) 9F09(2) 9F15(2) 9A(3) 9F21(3) 9F37(4) DF16(2) 9F1C(8)`, 37 bytes in total
GPO response	Format 2 (tag 77): AIP `1900`, AFL `08030301 10010200 18070800 20010100`
Records read (from AFL)	SFI 1 rec 3 · SFI 2 rec 1–2 · SFI 3 rec 7–8 · SFI 4 rec 1
Records signed for ODA	Only SFI 1 rec 3, plus the AIP (tag `9F4A` = `82`)
Network key	RuPay index `05`, 1984-bit, exponent 3 (public; its checksum matches)
Bank certificate `90`	248 bytes; bank key 1408-bit; exponent `9F32` = 03; no remainder (`92` empty or absent)
Card certificate `9F46`	176 bytes; card key 768-bit; exponent `9F47` = 03; no remainder (`9F48` absent)
Signature `9F4B`	96 bytes, read from SFI 4 rec 1 but generated at GPO. Format 05, 25 bytes of card dynamic data.
What the signature covers	SHA-1(signed data ‖ 9F37), where 9F37 is our random number for that tap. This is what makes it fresh.

Tests run against real card data

Real tap
With its own random number → PASS. All 4 steps verified.

Replay
Same signature with a different random number → FAIL.

Tamper
One byte of the signature changed → FAIL.

Demo cards and status

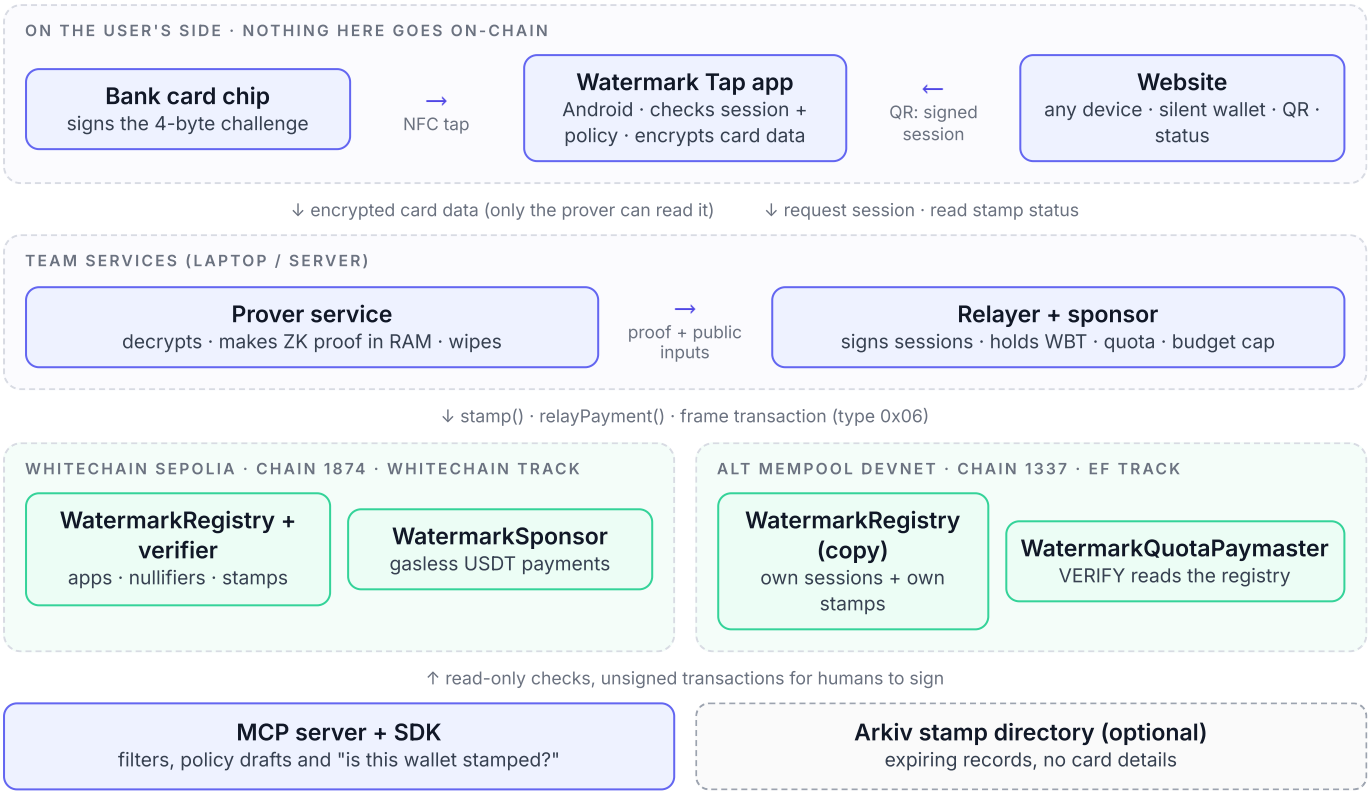
Card	Use	Status
SBI debit, RuPay	Live policy, shown to judges	**Must pass Card Test before kickoff.** The v2 record doesn't name which 2 cards passed. If its key sizes differ, the circuit needs that variant.
Pine Labs metro, RuPay ×2	Test policy only	Must never pass `watermark-live`. Used on stage to show a refusal.
Visa, Mastercard	Not in the demo	Untested on real cards. Say "implemented, not tested". Visa may sign at GPO; Mastercard at GENERATE AC.

What this means for the build

- All RSA exponents are 3, so each RSA check is just $x^2 \cdot x \bmod n$. About 12 SHA-1 blocks in total: bank certificate 4, card certificate 6, signature 2.
- The challenge is only 4 bytes (9F37). A wallet can't fit inside it, so binding happens through a signed session nonce (section F).
- A prepaid card also passes the maths. Debit-only needs the BIN filter (section I2).

C · SYSTEM ARCHITECTURE

The parts and how they connect



Part	Folder	Owner	Job
Watermark Tap	`/android`	Prashant	Opens a deep link, checks the session, reads the card, runs local checks, encrypts and sends card data
Website	`/web`	Prashant	Silent wallet, QR session, live status, gasless pay, sponsor rules, bot-attack demo
Prover service	`/prover`	Friend	Decrypts, makes the ZK proof, wipes card data, hands the proof to the relayer
ZK circuit	`/circuits`	Friend	Proves the EMV chain, freshness, filter, validity and nullifier (sections I, I2)
Contracts	`/contracts`	Friend	WatermarkRegistry + verifier, WatermarkSponsor, WatermarkQuotaPaymaster
Relayer	`/relayer`	Friend	Signs sessions, holds the WBT budget, pays gas only for valid proofs and stamped wallets
MCP + SDK	`/mcp /sdk`	Whoever is free first	Filters, policy drafts, stamp checks and unsigned transactions (section P)
Arkiv directory	`/arkiv`	Optional	Expiring stamp entities that the SDK and MCP can query

The full flow in one view

Stamp: website creates a wallet → relayer signs session → QR → app checks session and shows wallet → chip signs UN → phone checks maths and filter → encrypts to prover → ZK proof in RAM, wipe → relayer simulates → `stamp()` → website reads "Verified ✓" from the chain. **Pay:** business funds USDT → user signs → relayer checks stamp, quota, cap and budget → USDT settles, relayer pays gas. **EF:** second tap stamps on devnet → frame transaction with staked paymaster → public rules refuse, alt mempool admits.

What leaks, what doesn't, and what we say

The card doesn't encrypt anything. The chip **signs** our challenge with a private key that never leaves it, and its certificates are readable over NFC. Privacy comes from the ZK proof and from the encryption we add on the phone.

Data	Who can see it	Permanent?
Chip's private key	Nobody	Never leaves the chip
Card number, BIN, card key, certificates	Phone and prover only	No. In memory or RAM disk, then wiped. Users must trust the wipe; we can't prove it.
Card data on the network	Ciphertext only	Readable only with the prover's key, so a leaked key exposes captured traffic. Rotate it after the event.
Card expiry	Phone and prover only	No. On-chain we show only the "valid until" month.
Wallet, app ID, nullifier, stamp expiry, proof	Everyone	**Yes**, on-chain
App's filter (rules root and its public list)	Everyone	**Yes.** A stamp in an SBI-only app shows the wallet passed that filter.
USDT payments	Everyone	**Yes**, on-chain
IP address and timing	Relayer and prover	Only if logged, so request logs stay off in demo mode

Rules every part must follow

- **Never log** request bodies, card fields, witnesses or proofs' private inputs. No analytics or crash reporting anywhere in the flow.
- **One wallet per app.** Using one wallet in two apps links them, even though the nullifiers differ.
- **Warn before narrow filters.** The narrower the filter, the more a stamp reveals. The app and MCP must show this before a tap or a policy goes live.
- **Real ZK proofs only** (section I). Test that proofs were made in ZK mode.

Limits we state openly (README and to judges)

Limit	Plain explanation
Prover sees card data	The team's prover sees it briefly. On-device proving is the real fix, and it's on the roadmap.
One card, not one person	A person with three cards can get three stamps. A reissued card has a new key, so it gets a new stamp.
Possession only	A contactless tap has no PIN, so a stolen or borrowed card also works.
Nullifier isn't fully secret	Anyone who has read the card and knows the appId can compute it and link the card to the wallet.
Lost wallet	A stamp can't move to a new wallet yet in that app.
32-bit challenge	The chip's challenge is only 4 bytes, so sessions are rate-limited and the relayer chooses the nonce.
Trust in the relayer	The app's session signer could refuse a user or open too many sessions. It can't stamp without a real card.
Real launch	Needs PCI and legal review, network and bank terms, and a licensed BIN source. This hackathon build is testnet only.

One repo, both laptops

```
watermark/
├── CLAUDE.md                # rules for Claude Code (section E)
├── README.md                # setup, architecture, limits, track sections, disclosure of Card Test v2
├── .gitattributes           #  * text=auto eol=lf   (stops Windows/Mac line-ending fights)
├── .gitignore               # .env, node_modules, build/, out/, *.keystore, *.pem, card-logs/, target/, witness/
├── .env.example             # every variable, no real values
├── docs/  interfaces.md  AI_USAGE.md  prompts/  WHITECHAIN_FEEDBACK.md  EF_ALTMEMPOOL.md  PRIVACY.md
├── android/                 # Watermark Tap app (Kotlin or Java, Android Studio), package com.watermark.tap
├── web/                     # Next.js + viem + wagmi
├── prover/                  # Node/TS HTTP service calling nargo + bb
├── circuits/                # Noir project (or Circom fallback)
├── contracts/               # Foundry: src/ test/ script/
├── relayer/                 # Node/TS + viem
├── policies/                # test.json, live.json: public BIN-level lists + roots
├── sdk/   mcp/              # TypeScript
├── tools/testcard/          # synthetic EMV test card generator + fixtures (section S)
└── arkiv/                   # optional: schema.md friction.md reach.md why-arkiv.md how-you-use-arkiv.md
```

Use a fresh repo. The git root at `Desktop/watermark` today is the whole home folder, including `.ssh`. Create a new folder, run `git init` there, and confirm `git rev-parse --show-toplevel` prints that folder before any commit.

Prashant (Windows + Android)

- Git for Windows or GitHub Desktop
- Node.js 20 LTS + pnpm
- Android Studio (JDK 17 bundled), phone in USB-debugging mode
- Optional: WSL2 Ubuntu, if you also need to run nargo or bb

Friend (Mac)

- `xcode-select --install` (git), Node.js 20 LTS + pnpm
- Foundry: `curl -L https://foundry.paradigm.xyz | bash; foundryup`
- Noir: `noirup`; Barretenberg: `bbup` (pin versions and record them in the README)
- Workshop repo for the alt mempool (section L)

Environment variables (`.env.example`)

```
WHITECHAIN_RPC=https://rpc.testnet.whitechain.io/
WHITECHAIN_CHAIN_ID=1874
WHITECHAIN_EXPLORER=https://explorer.testnet.whitechain.io/
USDT_ADDRESS=0xcedfB56113143d55b2358b8CFbB0dEB5Ad2f55a7
RELAYER_PRIVATE_KEY=              # local only, funded with test WBT from the faucet
SESSION_SIGNER_KEY=               # local only, separate key that signs sessions
PROVER_DECRYPT_KEY_PATH=          # local only, outside the repo; public half is built into the app
REGISTRY_ADDRESS=   SPONSOR_ADDRESS=   DEVNET_REGISTRY_ADDRESS=   PAYMASTER_ADDRESS=
ALTMEMPOOL_EXEC_RPC=https://altmempool.forshtat.com
ALTMEMPOOL_SIDECAR_RPC=https://altmempool.forshtat.com/rpc      # check at kickoff; the workshop uses
altmempoolgateway.forshtat.com/rpc
ALTMEMPOOL_CHAIN_ID=1337
PROVER_URL=http://<laptop-lan-ip>:8787
ARKIV_RPC=https://rpc.rabdi.db-chain.testnet.arkiv.network/<accessKey>   # optional
ARKIV_CHAIN_ID=7794095
```

E · GIT WORKFLOW + CLAUDE.MD

How the two of you work in one repo

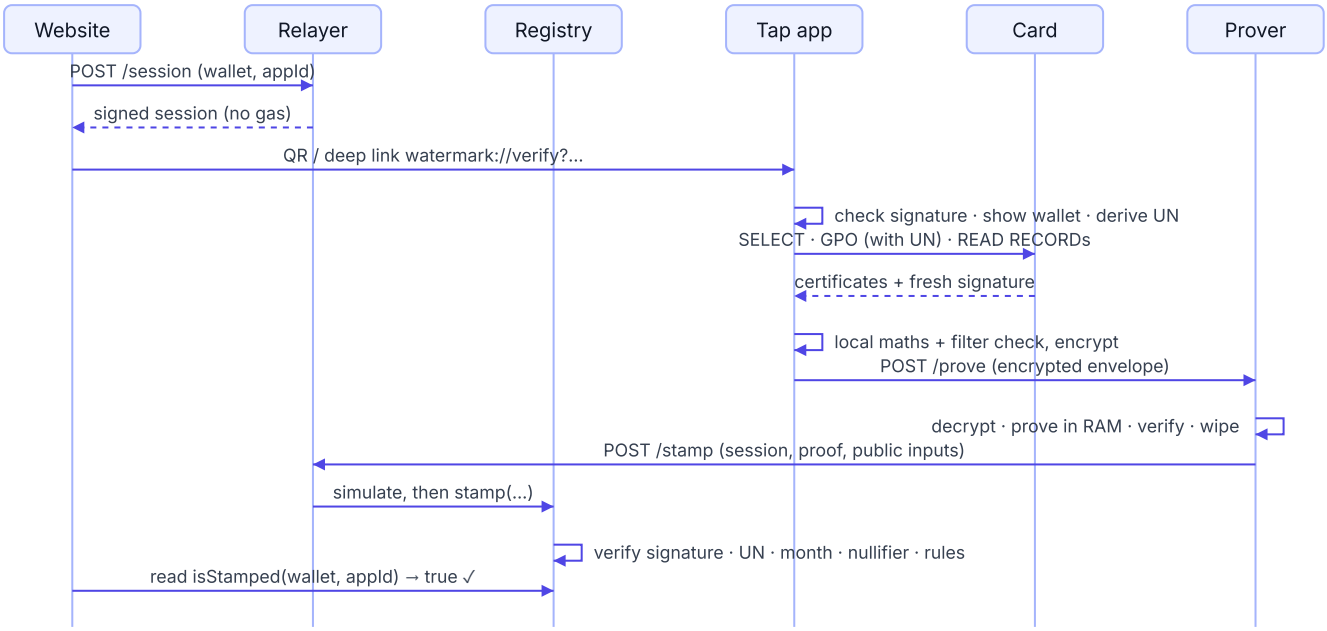
Paste this into CLAUDE.md at the repo root

```
# Watermark: rules for Claude Code
## Git (strict)
- NEVER run `git push`, on any branch or remote, for any reason. The human pushes manually.
- Never force-push, rebase pushed commits, `git reset --hard`, delete branches, or edit remotes.
- Only create a commit when the human asks. Keep commits small with clear messages.
- Before any commit, confirm `git rev-parse --show-toplevel` is this repo's folder.
- Before finishing a task: show `git status` and a short summary of changed files. Then stop.
## Data and secrets
- Never write real card data anywhere in the repo: no card numbers, raw APDU logs or real certificates.
- Never log request bodies, card fields or witnesses in any service.
- Use the synthetic test card in tools/testcard/ for all tests and fixtures.
- Never commit .env, private keys, keystores or access keys. Update .env.example instead.
## Scope
- Work only in the folder named in the task. Ask before touching another folder.
- docs/interfaces.md is the contract between parts. Don't change it without asking.
- Follow the build spec PDF section for the part you are building.
- The project is named Watermark. Use that name in all code and text.
## Hackathon rules
- All code is written after kickoff. Don't copy Card Test v2 code into this repo.
- Log AI help in docs/AI_USAGE.md (what you generated, which files, which prompt).
  Save planning prompts in docs/prompts/.
- Prefer simple, working code. Add a test for every parser or crypto function.
- Never fake a result. Label anything estimated or untested.
```

1. **Setup:** create a public repo with an MIT license. Add your friend under Settings → Collaborators. Commit only the skeleton, after kickoff.
2. **Branches:** `prashant/<topic>` and `friend/<topic>`. Merge into `main` with a pull request; the other person glances at it.
3. **Rhythm:** `git pull` before you start, commit every 30–60 minutes, and push before every break. Steady history is an ETHGlobal requirement.
4. **Ownership:** each person edits only their own folders. `docs/interfaces.md` changes need both people to agree.
5. **Pushing:** only Prashant or the friend runs `git push` by hand, after reviewing `git diff`. Claude Code prepares changes and stops.
6. **Spec files:** commit this PDF as a planning file. It's safe: it holds no BINs or card numbers.

The contract every part must match

Copy this section into `docs/interfaces.md` in hour 1. After that, both people can build in parallel.



One verification session, from QR to stamp. No gas is spent before step "stamp".

F1 · Session and QR (signed off-chain by the app's session signer)

```
Session = { wallet, appId, nonce (random bytes32), expiresAt (now + 600 s), validUntilMonth, chainId, registry }
sig     = EIP-712 signature by the app's sessionSigner, domain "Watermark", version 1, chainId, registry
UN      = first 4 bytes of SHA-256(the 32 raw nonce bytes)     # the 4-byte challenge (9F37) the card signs
month   = (YYYY - 2000) * 12 + (MM - 1)                        # validUntilMonth = current month + 3
QR text = watermark://verify?w=<wallet>&app=<appId>&n=<nonce>&e=<expiresAt>&v=<month>&c=<chainId>&r=<registry>&sig=
<sig>&p=<PROVER_URL>
```

The app holds the allowed signer address and refuses any session it didn't sign. Test scanning this QR by hour 3. If it's too dense, the QR carries a short ID and the app fetches the same signed session.

F2 · Card data (app → prover, encrypted, never stored)

```
Plain JSON: { "aid": "A0000005241010", "un": "<8 hex>", "caIndex": "05", "issuerCert": "<hex 90>", "issuerRem": "<hex
92 or empty>",
  "issuerExp": "03", "iccCert": "<hex 9F46>", "iccRem": "<hex 9F48 or empty>", "iccExp": "03", "staticData": "<hex:
ODA records per AFL + AIP>",
  "sdad": "<hex 9F4B>", "sigPath": "GPO_FDDA | GENAC_CDA | INTERNAL_AUTH", "termData": "<hex, starts with UN; extra
Visa fields; empty after UN for RuPay>" }
Sent as:    { "v": 1, "session": {...}, "sig": "0x..", "ek": "<AES key, RSA-OAEP-256>", "iv": "<12 bytes>", "ct": "
<AES-256-GCM>" }
            # AES-GCM additional data = the session's EIP-712 hash, so a ciphertext can't be moved to another session
```

F3 · Prover API

```
POST /prove   body: envelope from F2 → 202 { "jobId": "..." }   # checks signature, expiry and UN; one job per
session
GET  /prove/:jobId                  → { "status": "running|done|error", "txHash"?: "0x..", "error"?:
"NOT_IN_POLICY|EXPIRES_TOO_SOON|BAD_CARD|BUSY" }
# On done, the prover calls the relayer: POST /stamp. Errors never include card data.
```

F4 · Public inputs (order is fixed; circuit, contract and prover must match)

```
[0] caRoot          # Merkle root of allowed network CA keys (contract keeps the current root)
[1] rulesRoot       # Merkle root of this app's filter (section I2)
[2] appId           # 248-bit number, so it fits a BN254 field
[3] un              # the 4 bytes read as a big-endian number
[4] nullifier       # Poseidon2(pack(n_C), appId)
[5] validUntilMonth # the proof shows the card and bank certificates are valid through this month
# Every input is one bytes32 big-endian field element. Convert on the contract side. Never compare raw bytes4 with
bytes32.
```

F5 · Contract ABI (summary)

```
WatermarkRegistry
  registerApp(bytes32 salt, bytes32 rulesRoot, address sessionSigner) returns (uint256 appId)
      # appId = uint256(keccak256(abi.encode(msg.sender, salt))) >> 8. Rules are immutable: a new filter means a new
appId.
  stamp(Session s, bytes sessionSig, bytes proof, bytes32[] publicInputs)
  isStamped(address wallet, uint256 appId) view returns (bool ok, uint64 expiresAt)
  setCaRoot(bytes32 root)                                # owner only
  events: AppRegistered(appId, owner, rulesRoot, sessionSigner)   Stamped(wallet, appId, nullifier, expiresAt)
WatermarkSponsor (Whitechain)
  relayPayment(PaymentIntent intent, bytes userSig)      # relayer only; checks stamp, quota, budget
  quotaLeft(address wallet, uint256 appId) view returns (uint32)
WatermarkQuotaPaymaster (devnet)
  consume()                                              # called from the DEFAULT frame; see section L
```

F6 · Relayer HTTP and MCP tools

```
POST /session  { wallet, appId, chainId }                  → { session, sig, qr }     # no gas; rate-limited
POST /stamp    { session, sig, proof, publicInputs }       → { txHash }               # simulates first
POST /pay      { intent, signature }                       → { txHash }               # refuses: not stamped / quota
used / budget cap
GET  /status/:wallet?app=..&chain=..                        → { stamped, expiresAt, quotaLeft }   # reads the chain
GET  /policy/:appId                                        → { label, rulesRoot, leaves }        # public; the phone
recomputes the root
GET  /stats/:appId                                         → { budgetLeft, refusedByReason }     # counts only
MCP tools: section P
```

F7 · Payment intent (Whitechain)

```
Plan A: EIP-3009 transferWithAuthorization(from, to, value, validAfter, validBefore, nonce, v, r, s)
Plan B: EIP-2612 permit + transferFrom from WatermarkSponsor
Plan C: minimal WatermarkAccount smart account per user, execute(intent, sig) with EIP-712
# Selector check on 5 Nov found both Plan A and Plan B functions in the USDT implementation. Confirm with a real
transfer.
```

F8 · Policy file (`policies/*.json`)

```
{ "label": "SBI RuPay debit", "environment": "live | test",
  "leaves": [ { "network": "RUPAY", "networkId": 1, "prefixLength": 6, "prefix": "<digits>", "bank": "SBI",
                "type": "debit", "status": "tested | listed" } ],
  "rulesRoot": "0x..", "quotaPerDay": 20, "stampMonths": 3 }
networkId: 1 = RuPay, 2 = Visa, 3 = Mastercard
```

G · CARD LAYER (EMV OVER NFC)

Exactly what the phone says to the card

Card Test v2 showed this works on real RuPay cards. Write new code from this section after kickoff, and use v2 only as disclosed research.

#	Command (APDU)	Notes
1	`00 A4 04 00 0E "2PAY.SYS.DDF01" 00`	SELECT the payment directory (PPSE). List the AIDs from the `61` templates. If this fails, try a list of known AIDs.
2	`00 A4 04 00 Lc <AID> 00`	SELECT the app. Read the PDOL (`9F38`) and the label (`50` / `9F12`). The label is unsigned, so it's for display only.
3	`80 A8 00 00 Lc 83 L <PDOL data> 00`	GET PROCESSING OPTIONS. Put the session UN in `9F37` . Fixed values: `9F66` = 37004000 (then 36000000 or 3E004000 if refused), `9F02` = 000000000100, `9F1A` = `5F2A` = 0356, `9F33` = E0F0C8, `9F40` = 6000F0A001, `9F35` = 22, `9F1C` = "WMTEST01". Unknown tags are zeros.
4	`00 B2 <rec> <SFI<<3|4> 00`	READ RECORD for every record in the AFL (4-byte entries: SFI, first, last, ODA count). Collect the ODA static data while reading (section H).
5	`80 AE 90 00 Lc <CDOL1 data> 00`	Only if there's still no `9F4B` and the card has a CDOL1 (`8C`): GENERATE AC with ARQC + CDA (Mastercard path). Same UN. Not used on our RuPay cards.
6	`00 88 00 00 Lc <DDOL data> 00`	Last try: INTERNAL AUTHENTICATE (classic DDA), only if the AIP has the DDA bit. Default DDOL only at first (section I).

Handle status words `61xx` (send GET RESPONSE) and `6Cxx` (resend with the right Le). Set the IsoDep timeout to 5000 ms.

Nothing here is a payment: no authorisation message ever reaches a bank. The amount in `9F02` (1.00) is only a terminal value and is never charged. The card's transaction counter (ATC) does go up, so keep rehearsal taps on real cards few.

Network key IDs (RID): Visa `A000000003` , Mastercard `A000000004` , RuPay `A000000524` . Production CA keys are public, the same ones every payment terminal stores. Check each against its SHA-1 checksum, and credit the source in the README.

H · THE VERIFICATION MATHS, BYTE BY BYTE

The exact algorithm (the circuit proves this)

Notation: RSA(x, n) = x^3 mod n, output left-padded to the length of n. ‖ means concatenate. Offsets are 0-based, and L is the length of the recovered block. Reject any exponent other than 3.

Step 1 · Bank (issuer) certificate, with the network key

```
m = RSA(tag90, n_CA)                                  # L = 248 on RuPay (1984-bit CA)
check m[0]=0x6A, m[1]=0x02, m[L-1]=0xBC
check SHA1( m[1 .. L-22] ‖ tag92 ‖ tag9F32 ) == m[L-21 .. L-2]
NI = m[13]                                            # bank key length in bytes (176 on RuPay)
n_I = ( m[15 .. L-22] ‖ tag92 )[0 .. NI-1]
issuerId = m[2..5]  (leading digits of the card number, padded with F) · issuerCertExpiry = m[6..7] (MMYY)
```

Step 2 · Card (ICC) certificate, with the bank key

```
c = RSA(tag9F46, n_I)                                  # L = 176 on RuPay
check c[0]=0x6A, c[1]=0x04, c[L-1]=0xBC
check SHA1( c[1 .. L-22] ‖ tag9F48 ‖ tag9F47 ‖ staticData ) == c[L-21 .. L-2]
PAN = c[2..11] (BCD, F padding) · certExpiry = c[12..13] (MMYY) · NIC = c[19]   # 96 on RuPay
check issuerId digits are a prefix of PAN              # new in v3
n_C = ( c[21 .. L-22] ‖ tag9F48 )[0 .. NIC-1]
```

Step 3 · The card's signature, with the card key

```
s = RSA(tag9F4B, n_C)                                  # L = 96 on RuPay
check s[0]=0x6A, s[1] in {0x05 (DDA/fDDA), 0x95 (CDA)}, s[L-1]=0xBC
LDD = s[3]                                             # card dynamic data length (25 on RuPay)
```

Step 4 · Freshness (the anti-replay check)

```
check SHA1( s[1 .. L-22] ‖ termData ) == s[L-21 .. L-2], where termData starts with the UN:
  RuPay fast DDA, CDA, default DDA  : termData = UN
  Visa fast DDA                     : termData = UN ‖ 9F02 ‖ 5F2A ‖ 9F69
  INTERNAL AUTHENTICATE             : termData = DDOL data (default DDOL starts with UN)
```

Static data for ODA (input to Step 2)

```
for each AFL entry (SFI, first, last, odaCount):
  for the first odaCount records starting at `first`:
    if SFI ⩽ 10: append the record's value WITHOUT the 70 tag and length bytes
    else:        append the whole record, including tag and length
if tag 9F4A == 82: append the AIP (2 bytes)            # RuPay: 1 record (SFI 1 rec 3) + AIP = 183 bytes
```

I · ZK CIRCUIT SPEC

What the proof proves, and how

Stack choice (decide in the first 2 hours with a small spike)

Option	Pros	Cons
A · Noir + UltraHonk (preferred)	`noir-lang/noir-bignum` handles variable-size moduli (RuntimeBigNum). Fast laptop proving. `bb` writes a Solidity verifier.	On-chain verification costs more than Groth16 (measure it). SHA-1: `michaelelliot/noir-sha1` or write it yourself in about 80 lines.
B · Circom + Groth16	Cheapest on-chain check (~250k gas). Mature tooling (snarkjs).	Fixed-size circuits. The zk-email RSA code assumes exponent 65537 (adapt to 3). A setup run by one person can forge proofs, so disclose it.
C · zkVM (SP1 / RISC Zero)	Easiest to write.	Slow SHA-1 and big-integer proving. Fallback only.

Zero-knowledge must be on

Some UltraHonk modes aren't zero-knowledge, and a non-ZK proof could expose private inputs, including the card number. Flags differ by version: sources show `--zk` on older `bb`, and ZK by default with `--disable_zk` on 1.x. Pin one version, confirm it in that version's docs, generate the verifier in the same mode with `--oracle_hash keccak`, and record it in the README.

Circuit inputs

```
PRIVATE: issuerCert[248] issuerRem[] issuerExp · iccCert[176] iccRem[] iccExp · staticData[≤256] (+ length) ·
sdad[96] · termData[≤16] (+ length)
         n_CA[1984 bits] · networkId · ca path: caSiblings[6], caIndex · filter: prefixLength, ruleSiblings[12],
ruleIndex
PUBLIC:  caRoot, rulesRoot, appId, un, nullifier, validUntilMonth   (order from F4)
```

Constraints, in order

1. **CA key:** `Poseidon2(networkId, hash(n_CA))` is a leaf under `caRoot` (depth 6). With only RuPay keys in the tree, the network is still implied. Say so.
2. **Steps 1–4 from section H:** 3 RSA cubes, header and trailer bytes, and 3 SHA-1 checks (about 12 blocks). All exponents must equal 3.
3. **Issuer match:** the issuerId digits must be a prefix of the PAN.
4. **Freshness:** the first 4 bytes of `termData` equal `un`. Custom DDOLs that move the UN are out of scope for now.
5. **Filter:** section I2. The PAN comes from the signed certificate, never from an unsigned tag.
6. **Validity:** certExpiry and issuerCertExpiry, each converted to a month number, are ≥ `validUntilMonth`.
7. **Nullifier:** `nullifier = Poseidon2(pack(n_C), appId)`. It uses the card key, never the card number, because card numbers can be guessed from leaked lists.
8. **Never output** the PAN, BIN, expiry, network, certificates or keys.

Sizes to target first (from our real cards)

CA 1984-bit → bank 1408-bit → card 768-bit, exponents 3, staticData 183 bytes, signature 96 bytes. Build a variant fixed to these RuPay sizes and make it pass. Confirm the SBI card's sizes before kickoff. Add variants (or max-size padding) only if time allows, and don't advertise Visa or Mastercard as working until a real card passes.

Tip for Claude Code: write the circuit test against the synthetic test card first (section S), then against one real card on the owner's laptop only. Never put a real card's witness in the repo.

I2 · FILTERS

Bank, network and card type filters

The chip has no "SBI" or "debit" field. Network is proven by the key that signed the card. Everything else comes from the card's BIN and a list we maintain.

Filter	How it's enforced	Accuracy
Network	CA key under `caRoot`, tied to the leaf's `networkId`	Cryptographic
Bank, type, country	BIN prefix in the app's allowlist	Only as good as the list
Name, balance, age, KYC	Not possible	Never claim these

```
leaf      = Poseidon2(networkId, prefixLength, prefix)     # prefixLength is 6 or 8; prefix = first digits of the PAN
from the certificate
rulesRoot = Merkle root of leaves (Poseidon2, depth 12)    # example: SBI RuPay debit, or Axis Mastercard
```

- **The phone checks first** using `GET /policy/:appId`, after recomputing the root. A refused card's data never reaches the prover.
- **Each leaf has a status:** `tested` means a real card passed; `listed` means it came from a list and hasn't been proven. There's no complete free BIN list, so don't claim "all SBI cards".
- **Apps:** `watermark-test` allows Pine Labs and synthetic cards. `watermark-live` allows only the tested SBI RuPay debit prefix, and it has a new appId for each demo run.

Three small contracts

WatermarkRegistry.sol (Whitechain Sepolia, and a copy on the alt mempool devnet)

```
struct App { address owner; bytes32 rulesRoot; address sessionSigner; }
struct Session { address wallet; uint256 appId; bytes32 nonce; uint64 expiresAt; uint32 validUntilMonth; uint256
chainId; address registry; }
mapping(uint256 ⇒ App) apps;                  mapping(bytes32 ⇒ bool) sessionUsed;
mapping(bytes32 ⇒ address) nullifierWallet;   bytes32 caRoot;
mapping(address ⇒ mapping(uint256 ⇒ uint64)) stampExpiry;   # wallet ⇒ appId ⇒ expiry (0 = none)

stamp(s, sessionSig, proof, publicInputs):
  app = apps[s.appId]; require(app.owner ≠ 0)
  require(block.timestamp ⩽ s.expiresAt && s.chainId == block.chainid && s.registry == address(this))
  h = eip712Hash(s); require(!sessionUsed[h] && recover(h, sessionSig) == app.sessionSigner)
  require(pi.caRoot == caRoot && pi.rulesRoot == app.rulesRoot && pi.appId == s.appId)
  require(pi.un == uint32(bytes4(sha256(abi.encodePacked(s.nonce)))))
  m = currentMonth(block.timestamp)
  require(pi.validUntilMonth == s.validUntilMonth && m + 1 ⩽ pi.validUntilMonth && pi.validUntilMonth ⩽ m + 3)
  owner = nullifierWallet[pi.nullifier]; require(owner == address(0) || owner == s.wallet)   # same wallet may renew
  require(verifier.verify(proof, publicInputs))
  nullifierWallet[pi.nullifier] = s.wallet; sessionUsed[h] = true
  stampExpiry[s.wallet][s.appId] = endOfMonth(pi.validUntilMonth)     # stamp goes to s.wallet, never msg.sender
  emit Stamped(s.wallet, s.appId, pi.nullifier, expiry)
```

Because the stamp always goes to the session's wallet, a stolen proof copied from the mempool is useless to anyone else. Nullifier use is per registry, so one card can stamp once per app on each chain.

WatermarkSponsor.sol + gasless USDT (Whitechain track)

Verified 5 Nov: the test USDT is a proxy at `0xcedf…55a7`, and its implementation contains the `permit` and `transferWithAuthorization` selectors. Calling `DOMAIN_SEPARATOR()`, `nonces()` and `authorizationState()` succeeds. No transfer has been tested yet.

1. **Plan A:** the user signs EIP-3009. The relayer calls `relayPayment`, which checks the stamp, quota and budget, then calls `transferWithAuthorization`.
2. **Plan B:** EIP-2612 permit, then `transferFrom`.
3. **Plan C:** a minimal `WatermarkAccount` smart account per user.

USDT supply (check in hour 0): the faucet docs mention WBT only, and self-minted tokens don't count. Ask Whitechain how to get the official test USDT. A business wallet then funds each demo wallet, so the user does nothing. The user never holds WBT and never sees gas.

Verifier.sol

Generated by `bb write_solidity_verifier` (Noir) or snarkjs (Groth16). Deploy it, then pass its address to WatermarkRegistry. Check it fits the contract size limit. Measure its gas and write the number in the README. Deploy with `forge script`, verify on the Whitechain explorer (check the method at kickoff), and paste explorer links into the README.

`currentMonth` and `endOfMonth`

Use a small audited date library or a 15-line civil-date conversion, and unit-test the month and year boundaries. Whitechain and devnet must use the same code.

K · RELAYER AND DRAIN PROTECTION

The answer to "what stops someone draining it?"

Layer	Rule
0 · No gas before proof	Sessions are signed off-chain. `/session` and `/prove` are rate-limited per IP and queue-limited.
1 · Stamp gate	Stamp transactions are sent only after the prover verifies the proof and the relayer simulates the call.
2 · Payment gate	No stamp for this app → refused before any gas is spent (checked off-chain first, then again on-chain).
3 · Daily quota	N sponsored transactions per stamp per day (20 in the demo). The counter lives in WatermarkSponsor.
4 · Per-transaction gas cap	The relayer simulates first and refuses any transaction above a gas limit.
5 · Budget cap	A total daily WBT budget, with a separate stamp budget. When hit, the relayer pauses and the dashboard shows it.
6 · One card, one wallet	Nullifiers stop the same card from stamping a second wallet in this app.

How the relayer gets its WBT

Testnet: from the faucet at `faucet.testnet.whitechain.io`. It gives **0.5 WBT per rolling 24 hours** and needs a GitHub account at least 30 days old. Limits apply per wallet, IP and GitHub account, so claim early and ask Whitechain if you need more. Gas price was about 5 gwei on 5 Nov.

Production story: the business tops up the relayer wallet, and the dashboard shows "budget left today". Put this in the README and `WHITECHAIN_FEEDBACK.md`, because the track asks for it.

Relayer build notes

- Node + TypeScript + viem. Two keys loaded from `.env`, never committed: relayer and session signer.
- Endpoints from F6. Serialise transactions to avoid nonce clashes.
- Log every refusal with only its reason and a truncated wallet. The demo shows this log.

Bot script for the demo (`relayer/scripts/bots.ts`)

It creates 1,000 wallets and sends 1,000 requests to `/pay`.

Protection off

25 real sponsored transactions, measured. The other 975 are an estimate from that gas, labelled **"estimated"** on screen. A full real run would cost about 0.33 WBT (1,000 × a rough 65k gas × 5 gwei).

Protection on

All 1,000 are really sent and really refused off-chain. Budget is unchanged.

L · EF ALT MEMPOOL (EIP-8141)

Why this needs the alt mempool

EIP-8141 adds frame transactions (type `0x06`). A VERIFY frame runs like a STATICCALL and ends with APPROVE. The public mempool caps verification at `MAX_VERIFY_GAS` = 100,000 and limits state access. The alternative mempool (ERC-8421) allows more, using staking and reputation.

Must-have · Quota paymaster

A `WatermarkQuotaPaymaster` whose VERIFY frame (flags `0x1`) reads WatermarkRegistry (is `tx.sender` stamped and not expired? any quota left?), checks the transaction contains the consume frame, then calls `APPROVE(APPROVE_PAYMENT)`. Rule STORAGE-032 lets a **staked** entity read any storage in a non-entity contract.

Stretch · Proof check inside VERIFY

Verify the ZK proof in VERIFY so the first sponsored transaction also stamps. Staked budget: **1,000,000 gas** in the draft rules, 2,500,000 marked TBD on another docs page. Only try this if the measured verifier fits. Groth16 (~250k) would; UltraHonk may not.

Steps

1. Clone `github.com/forshtat/mempool-workshop` (Node 20+, yarn v1, Foundry). Run `yarn step_1_check`, then step 3 (paymaster-sponsored transaction), step 7 (stake) and step 8 (alt mempool accepts). This proves the toolchain on day 1.
2. Deploy a copy of WatermarkRegistry and its verifier on the devnet (chain 1337). Register an app there.
3. **Stamp with a second tap.** A proof is tied to one session and one registry, so open a session with `c=1337` and tap again. Same card, same flow, separate registry.
4. Write WatermarkQuotaPaymaster (start from the workshop's SponsorPaymaster). Stake it with `addStake(86400)` on the NaiveStakingRegistry `0xd51c04d7a0b01124e84de152f4e56d9eecb22d2f`, sending more than the 0.01 ETH minimum. Use your own key, never the shared one.
5. Build the frame transaction: `[VERIFY → paymaster, flags 0x1] [SENDER → the user's call] [DEFAULT → paymaster.consume()]`. Submit with `eth_sendFrameTransaction` to the sidecar.
6. **Protect the quota.** VERIFY is static, so the counter updates in the DEFAULT frame. VERIFY must reject a transaction whose frames don't match this shape, and must give `consume()` enough gas. Ask the alt mempool team how best to read frames in VERIFY, and confirm what a reverted frame does.
7. Test attacks: no stamp, expired stamp, missing consume frame, quota used up. Each must be refused.
8. Show it's refused by the canonical rules (`eth_sendRawTransaction` to ethrex) and accepted by the alt mempool. Save both results and the explorer link in `docs/EF_ALTMEMPOOL.md`, with the design and trade-offs.

Devnet facts (from the alt mempool docs; check at kickoff)

Execution RPC `https://altmempool.forshtat.com` · sidecar RPC `/rpc` (the workshop README uses `altmempoolgateway.forshtat.com/rpc`) · chain ID 1337 · 6-second blocks · explorer `explorer.altmempool.wtf`. No faucet: fund your own key once from the shared genesis account. New-account transfers need about 250k gas. The devnet can be reset at any time, so keep deploy scripts re-runnable. The rules are a draft and may change. The alt mempool team is at the venue: meet them early.

M · ANDROID WATERMARK TAP APP

One screen, one job

New code, package `com.watermark.tap`. minSdk 21, targetSdk 34, permission `NFC`, feature `android.hardware.nfc`. `allowBackup=false`. `FLAG_SECURE` on the tap screen except in the mirrored demo build. Build with Android Studio on Windows, with a keystore that stays out of git. Never write card data to disk, logs, clipboard or analytics. The screen never shows the card number or BIN.

1. **Open from a deep link:** intent filter for scheme `watermark`, host `verify`. Scan with the normal camera app, and test this on the demo phone by hour 3. Fallback: open the website on the phone and tap the same link. Parse `w, app, n, e, v, c, r, sig, p`.
2. **Check the session:** recover the signer and compare it with the address built into the app. Check expiry. If anything's wrong, stop.
3. **Show consent:** app name, filter, what the stamp will reveal, the wallet's last 4 characters and a countdown. "Check this matches the website, then tap your card."
4. **Compute the UN:** first 4 bytes of SHA-256 of the 32 raw nonce bytes.
5. **Read the card:** reader mode (NFC-A/B, skip NDEF), then the flow from section G, with the session UN in `9F37`.
6. **Check locally:** run the chain and freshness maths, then the policy from `GET /policy/:appId` (recompute the root). If it fails, show why and stop: nothing is sent.
7. **Encrypt and send:** F2 envelope to `p` with the built-in prover public key. Add a network security config for the plain-HTTP LAN host. Clear card data from memory as far as the runtime allows.
8. **Show progress:** "Proving…" → "Stamped ✓" (read from `/status`, which reads the chain) → a button back to the website.

N · WEBSITE (NEXT.JS)

Pages and flows

Page	What it does
`/`	One-line pitch, "Bots can't hold plastic", and buttons for the demo pages.
`/verify`	Silently creates a browser wallet for this app, with no crypto steps. Connecting MetaMask is hidden under "advanced". → `POST /session` → big QR plus the `watermark://` link → polls `/status` → "Verified ✓" with the Whitechain explorer link. Shows the wallet's last 4 characters to compare with the phone.
`/pay`	Gasless USDT send: amount + recipient → signs in the background → `POST /pay` → explorer link showing gas paid by the relayer. The payer never sees gas, seed phrases or a chain picker.
`/attack`	Big "Start bot attack" button. Live counters: requests, refused, sponsor budget left, plus an "estimated" label when protection is off. Toggle protection on and off.
`/sponsor`	The app's rules (networks, banks, type, quota), TEST or LIVE label, Merkle root, what a stamp reveals, and budget and quota stats.
`/dev`	How to integrate: contract call, SDK snippet, MCP config.

Stack: Next.js (App Router), viem + wagmi, Tailwind. No trackers or analytics. Works on any device, including the friend's iPhone. Only the card tap needs an Android phone. On stage: laptop at 150% zoom, QR large enough to scan from 30 cm.

O · PROVER SERVICE

Runs on the team laptop

- Node + TypeScript HTTP server on port 8787, implementing F3. It runs `nargo execute` (witness) and `bb prove` (or snarkjs), verifies the proof locally, then POSTs it to the relayer.
- **Before any work:** check the session signature, expiry and UN, and that this session has no job yet. Run one proof at a time, with a short queue.
- **Card data handling:** decrypt in memory. Write the `Prover.toml` and witness only to a RAM disk, and delete them right after proving. Never log. Node can't guarantee memory is cleared, so state this as best effort.
- The decrypt key lives outside the repo. Generate it after kickoff and rotate it after the event.
- Precompute anything public: Barrett parameters for the network key and the Merkle trees for each app.
- Measure and print the proving time. It goes into the README and the demo ("proof in N seconds").
- Run it on the same Wi-Fi or hotspot as the phone. Print its LAN URL at startup; the website puts it in the QR.

On-device proving is the real privacy fix, because then no server sees card data. It's a roadmap item, not a promise for this build. Anyone can also self-host the prover, which moves that trust to them.

Q · ARKIV (OPTIONAL THIRD TRACK)

Expiring stamp directory

Add this only if sections J–N work by Friday evening. It publishes another copy of stamp data, so keep it minimal.

- **Network:** Arkiv "Rabdi" testnet, chain ID 7794095, `@arkiv-network/sdk` 1.0.0. The RPC URL carries an access key (`hub.arkiv.network/access-keys`). Confirm at the Arkiv booth.
- **Write:** on every `Stamped` event, the relayer creates an entity `{ type: "watermark-stamp", app, wallet, expiresAt }` , expiring with the stamp. No network or card fields.
- **Read:** the SDK and MCP query `type = "watermark-stamp" && app = "…" && wallet = "0x…"` , and fall back to the contract.
- **Judging files (25% each):** `friction.md` , `reach.md` , `why-arkiv.md` , `how-you-use-arkiv.md` . Also required: `schema.md` , plus the writer address and one runnable query in the README. Answer "isn't this a copy of on-chain state?" honestly in `why-arkiv.md` .

Filters and checks for apps and AI agents

Rule: the MCP never takes card data and never holds keys. It only reads data, or prepares unsigned transactions for a person to review and sign. No tool accepts a card number or BIN from a user's own card.

Tool	What it does
`watermark_search_issuers`	Lists supported banks, networks and card types, each marked `tested` or `listed`
`watermark_draft_policy`	Turns "SBI RuPay debit only" into BIN leaves and a root, with warnings about untested leaves and narrow filters
`watermark_explain_policy`	Explains who passes and what a stamp reveals publicly
`watermark_prepare_register_app`	Returns an unsigned `registerApp` transaction for the app owner
`watermark_check_wallet`	Stamped or not, plus expiry, for one wallet and app
`watermark_check_wallets`	The same for a list of up to 200 wallets, for airdrops and allowlists
`watermark_create_verify_link`	Creates a verification link for a wallet and app
`watermark_get_policy` / `watermark_get_quota`	Returns an app's rules and a wallet's remaining quota
`watermark_budget_status`	Returns budget left and refusal counts, with no wallet list
`watermark_prepare_reward`	Prepares a USDT payout to stamped wallets, for the business to sign. Stretch goal.
`watermark_integration_snippet`	Generates Solidity or TypeScript integration code

```
// sdk/src/index.ts
export async function check(wallet, appId, chain = "whitechain"): Promise<{stamped:boolean, expiresAt:number}>
export async function checkMany(wallets, appId, chain = "whitechain"): Promise<Record<string,{stamped:boolean,
expiresAt:number}>>
export function verifyLink(appId, wallet, siteUrl): string
export async function getPolicy(appId): Promise<{label:string, rulesRoot:string, quotaPerDay:number}>
export function draftPolicy({networks, banks, types, quotaPerDay}): Policy
```

The MCP server uses `@modelcontextprotocol/sdk` over stdio. Ship a ready `claude_desktop_config.json` snippet in `mcp/README.md`. Tool results are data, so treat text in them as untrusted, and keep every write behind a human signature.

Real use cases to offer

- **Airdrops and faucets** that bots can't farm
- **Gasless apps** protected from draining
- **One-per-card drops:** tickets or NFTs, with fewer scalpers
- **Bank-specific offers:** "SBI debit holders get this reward" without collecting card details
- **AI agents:** check a wallet before paying a reward or giving access

Track note: the MCP has no prize of its own. Whitechain doesn't accept AI chats that never move money, so the website payment stays the main flow. Demo line: "An AI agent asks Watermark before sending rewards to a wallet." Don't sell the MCP as KYC, age checks or one-person-one-vote.

Who does what, and when

Prashant · Windows + Android

- Watermark Tap app (section M); the only person who can tap cards
- Website (section N)
- Card testing, and borrowing a Visa or Mastercard if possible
- Demo, video, README, `WHITECHAIN_FEEDBACK.md`, pitch
- Reviews and pushes all commits

Friend · Mac (+ iPhone for web testing)

- ZK circuit (sections I, I2) and prover (section O)
- Contracts (section J) and relayer (section K)
- Alt mempool work (section L)
- Synthetic test card (section S)

Before kickoff (no project code)

1. **Confirm** the kickoff time, and send the organisers the question from page 2.
2. **Prashant:** run the SBI debit card through Card Test. Share only PASS or FAIL, key sizes and AIP/AFL. Never share the number, BIN or raw log.
3. **Both:** install tools, and find the Whitechain and alt mempool teams.

Checkpoints (hours from kickoff)

Hour	Prashant	Friend	Done when
0–1	Fresh repo, CLAUDE.md	`docs/interfaces.md` (section F)	First commit on main
0–3	Get USDT, test a gasless transfer, test QR scan	Alt mempool workshop steps 1, 3, 7, 8; Noir vs Circom spike; pin `bb`	Accepted transaction; stack chosen
3–6	New card reader, deep link, session check, website QR	Synthetic test card, relayer skeleton, signed sessions	QR opens the app; session verifies
6–10	App encryption, policy check, send to prover	Circuit and prover	Test card's proof verifies; proof time logged
10–20	`/verify` and `/pay` pages	Contracts on Whitechain, relayer rules	QR → tap → stamp → USDT paid; explorer links in README
20–26	`/attack` and `/sponsor`; real card tests	Quota paymaster, devnet stamp	Admitted transaction link
26–28	README, `WHITECHAIN_FEEDBACK.md`, `PRIVACY.md`	SDK and MCP; Arkiv only if all else works	Agent check works
28–30	Rehearse	`docs/EF_ALTMEMPOOL.md`	Demo ready to run
30–33	Feature freeze; record the video	Bug fixes only	Three clean demo runs
33–35	Run the submission checklist	Check the repo and every link	Everything linked
By 8 AM Sat	**Submit an hour early** (finalist + partner prizes)		Submission confirmation

Sit together for 10 minutes every 3–4 hours. If something blocks for more than an hour, drop to the fallback in section V. If the ZK circuit isn't working by hour 20, switch to attester mode and say so openly.

Test without real card data in the repo

Synthetic test card (`tools/testcard/`)

1. Generate RSA keys with exponent 3 at the real sizes: a test CA (1984-bit), a test bank (1408-bit) and a test card (768-bit).
2. Build the bank and card certificates in the exact EMV formats from section H (headers `6A 02` / `6A 04`, trailer `BC`, SHA-1 hashes, static data = one record + AIP), using a fake debit BIN.
3. Sign a format-05 signature over a given UN, with 25 bytes of dynamic data.
4. Output the F2 JSON plus the test CA modulus. Commit these fixtures. They contain no real data.

Tests to write

Test	Expected
Synthetic card, correct UN	Circuit, prover and contract all pass
Same signature, different UN (replay)	Proof fails, or contract rejects on UN
One flipped byte in any certificate or the signature	Fails
Exponent other than 3, or issuerId not matching the PAN	Fails
Same card, second session, different wallet, same app	Contract rejects it (card already used)
Same card, same wallet, later session	Passes as a renewal
Same card, different app	Passes, with a different nullifier
Same session used twice	Contract rejects it
Session from the wrong signer, chain or registry, or expired	Refused by app, prover and contract
BIN not in the filter; network doesn't match the leaf	Phone refuses; a forced proof fails
Card or bank certificate expiring before `validUntilMonth`	Fails with `EXPIRES_TOO_SOON`
`validUntilMonth` **outside the allowed window; month and year boundaries**	Contract rejects / computes correctly
`appId`**, roots and** `un` **encoding across circuit and contract**	All match; values fit the field
Proof submitted by a stranger	Stamp still goes to the session's wallet
Fake QR with attacker's prover URL or unsigned session	App stops, or attacker gets only ciphertext
Ciphertext moved to another session	Decryption fails
Search logs, temp folders and app storage for fixture card data after a run	Not found
Proof-mode check	Confirmed ZK; verifier built in the same mode
Relayer: no stamp / quota used / budget cap / `/session` **spam**	Refused, reason logged, no gas spent
Paymaster: no stamp / expired / missing consume frame / no quota	Refused
Real USDT payment using the official address	Explorer shows the relayer paid gas

Real cards (owner's laptop only)

SBI debit RuPay	Passes `watermark-live`. Record "SBI debit RuPay: tested" in the README, only after it really passes.
Pine Labs metro ×2	Passes `watermark-test`; refused on `watermark-live`, with no data sent.

The behavioural reference for maths tests is Card Test v2's real-card, replay and tamper results. Keep real-card checks off the repo.

T · DEMO SCRIPT

4 minutes on stage

0:00	**Hook:** "This script made 1,000 wallets." Run `/attack` with protection off: 25 real sends drain the budget and the rest are labelled estimates.	PROBLEM
0:35	**Filter:** tap a Pine Labs metro card on the live app: refused, nothing sent. "A genuine card, but not allowed by this app's rule."	FILTER
0:55	**Verify:** laptop shows the QR → scan → phone shows the same wallet → tap the SBI debit card → "Verified ✓" with the Whitechain explorer link.	STAMP
1:40	**Pay:** gasless USDT on `/pay`. The explorer shows gas paid by the relayer and official USDT settled.	WHITECHAIN
2:10	**Same card, new wallet:** "Card already used in this app."	ONE CARD
2:30	**Bots again, protection on:** 1,000 really refused, budget unchanged.	DRAIN-PROOF
2:55	**Privacy:** the explorer shows only a random code. No card number, BIN or card expiry. Then say plainly: "our prover sees card data briefly; on-device proving is next."	PRIVATE
3:20	**Alt mempool:** the quota paymaster transaction, refused by the public rules and admitted by the alt mempool (explorer link). Its stamp came from an earlier real tap; say so.	EF
3:45	**Agent:** one MCP check on the stamped wallet, then close.	MCP

Stage kit

- SBI debit, both Pine Labs cards, and the Android phone with a backup
- **A fresh live appId and wallet for each run.** A card stamps only once per app, so a rehearsal can block the live demo.
- Prover running, relayer funded, devnet scripts re-runnable, devnet stamp done beforehand
- Phone hotspot (venue Wi-Fi may block LAN traffic)
- Recorded backup video in a browser tab

Demo video rules

- 2–4 minutes, at least 720p, not sped up; intro under about 20 seconds
- Your own voice (no AI voiceover); not recorded on a phone; no more than 4 bullets per slide
- Show the tap with screen mirroring (scrcpy works on Windows)
- Whitechain wants every party shown: user, business, relayer
- **Hide the card number** on camera; never show it or the BIN

Be ready for these judge questions

Question	True answer
Does the card number leave the phone?	Yes, encrypted to our prover, briefly. It's never on-chain. On-device proving is the fix.
Is this one person, one stamp?	No. One card, one stamp per app. It raises a bot's cost; it isn't identity.
What about a stolen card?	It would pass, because a tap has no PIN.
Does Visa or Mastercard work?	Say what's true on the day. As of this spec: untested on real cards.
How do you know a card is SBI debit?	From its signed BIN and our list. The chip has no bank field.
Why the alt mempool?	The paymaster reads a registry during validation, which the public mempool refuses.
Why on-chain stablecoins?	Any app can check stamps and settle USDT with no shared card network or merchant account, and the business pays gas.

U · SUBMISSION CHECKLIST

Tick every box before 8:00 AM Saturday

Hard deadline: 9:00 AM IST. Late submissions aren't accepted.

ETHGlobal (all)

- ☐ Public repo, open-source license, steady commit history
- ☐ All code written after kickoff; reused libraries listed in README
- ☐ Card Test v2 and pre-kickoff planning disclosed; organisers' answer followed
- ☐ `docs/AI_USAGE.md` + prompts and spec files (this PDF counts as planning)
- ☐ Demo video meets the rules (section T)
- ☐ Up to 3 partner prizes selected, each with "how we used it" + feedback
- ☐ Opt in to Finalist judging (4 min demo + 3 min Q&A)
- ☐ Card number, BIN and raw logs searched for and absent from the repo and video

Whitechain · Scale

- ☐ Contracts deployed and verified on Whitechain Sepolia (1874)
- ☐ A payment in official USDT settled in the main flow, explorer link in README
- ☐ Payer flow has zero crypto steps
- ☐ README line: why on-chain stablecoin settlement is needed
- ☐ README: how the business pays gas, where WBT comes from, drain protection
- ☐ `WHITECHAIN_FEEDBACK.md` (what worked, where we got stuck, doc gaps)
- ☐ README sections on setup, architecture and current limitations

EF · Alt mempool

- ☐ Admitted transaction link on the devnet explorer, or the code that submits it
- ☐ `docs/EF_ALTMEMPOOL.md`: why the public mempool refuses it, design and trade-offs
- ☐ Alt mempool selected as our EF track (placed in at most one EF prize)

Arkiv (if added)

- ☐ Writes on testnet plus a user-visible read
- ☐ `/arkiv/`: schema, friction, reach, why-arkiv, how-you-use-arkiv
- ☐ Writer address and a runnable query in README

V · RISKS AND FALLBACKS

If something breaks

Risk	Fallback
Organisers say the prior work is a problem	Follow their ruling. Rewrite as planned, or ask whether a Continuity route applies.
SBI card fails Card Test	Show the Pine Labs cards on the test policy only, and say clearly that the live SBI policy is unproven. Try borrowing another debit card.
The circuit isn't ready by hour 20	Ship the stamp with the full maths check in the prover, signed by it (an "attester" mode). Say so honestly; ZK is the next step. Keep both paths behind a flag.
On-chain verifier too expensive or too big	Switch to Groth16 (Circom) and disclose its setup. Measure gas early.
Official USDT not obtainable	Ask Whitechain at the booth. Don't substitute a self-minted token: it doesn't count.
USDT authorization fails	Plan B (permit), then Plan C (smart account). See F7.
Alt mempool devnet resets or breaks	Re-run the deploy scripts. Keep a recorded admitted transaction and the submitting code (the track accepts code).
Venue Wi-Fi blocks phone → laptop	Phone hotspot. Last resort: the relayer forwards the same ciphertext over HTTPS.
QR too dense, or camera won't open the link	Short session ID in the QR, or open the link from the website on the phone.
Live tap fails on stage	Second card → second phone → backup video. Never fake a step.

W · LINKS

Everything in one place

1. **ETHGlobal Mumbai prizes and rules:** `https://ethglobal.com/events/mumbai/prizes` · `https://ethglobal.com/events/mumbai/info/details`
2. **Whitechain Sepolia:** RPC `https://rpc.testnet.whitechain.io/` · chain 1874 · WBT · explorer `https://explorer.testnet.whitechain.io/` · faucet `https://faucet.testnet.whitechain.io` · docs `https://docs.whitechain.io`
3. **EIP-8141 frame transactions:** `https://eips.ethereum.org/EIPS/eip-8141` · ERC-7562: `https://eips.ethereum.org/EIPS/eip-7562`
4. **Alt mempool:** `https://docs.altmempool.wtf` · devnet `/guides/devnet` · staking `/guides/staking/` · budgets `/rules/budget/` · storage `/rules/storage/` · explorer `https://explorer.altmempool.wtf` · workshop `https://github.com/forshtat/mempool-workshop`
5. **Noir:** `https://github.com/noir-lang/noir-bignum` · params `https://github.com/noir-lang/noir-bignum-paramgen` · RSA `https://github.com/noir-lang/noir_rsa` · SHA-1 `https://github.com/michaelelliot/noir-sha1`
6. **Barretenberg:** `https://barretenberg.aztec.network/docs/bb-cli-reference/` · `https://barretenberg.aztec.network/docs/how_to_guides/how-to-solidity-verifier/`
7. **Arkiv Mumbai workshop** (Rabdi testnet, SDK 1.0.0): `https://github.com/SantiagoDevRel/mumbai-ethglobal` · access keys `https://hub.arkiv.network/access-keys`
8. **Public CA keys:** proxmark3 `capk.txt` (Visa, Mastercard) and the RuPay production list in `mmors-1/nfc10` `capk.xml`. Check every key against its SHA-1 checksum, and check each source's license before including it.
9. **Prior art to cite in the README:** Carte Bleue (ETHGlobal Lisbon 2026), zkEMV (ZK Hack Berlin 2025), GOAT RAMP (ETHGlobal Istanbul).

Still open on 5 Nov

- Kickoff time: not confirmed (the official event page wasn't loading). Assumed about 36 hours before the deadline.
- Organisers' answer on pre-kickoff work.
- SBI debit Card Test result and key sizes.
- Source of official test USDT, and a real authorization transfer.
- Exact `bb` flags for the pinned version; the Whitechain verification method.
- How VERIFY reads frames, and what a reverted frame does (ask the alt mempool team).

Status of this document: v3 is a design. Real-card results in section B come from v2's Card Test work. The changes in v3 have not been built or run yet, so treat the code-level details as the plan to test, not as proven.